

LAHORE GARRISON UNIVERSITY

Department of Computer Science

PROJECT DOCUMENTATION

CUDA-MPI MatMul Engine

Distributed Matrix Multiplication Framework

Project Title: CUDA-MPI MatMul Engine  
Semester: 6th Semester, Fall-23  
Course: Parallel & Distributed Computing Lab  
Instructor: Miss Ifra Shahzad  
Date: May 2026  
University: Lahore Garrison University

Team Members:

| # | Name           | Roll No.         |
|---|----------------|------------------|
| 1 | Muhammad Fezan | Fall-23-BSCS-466 |
| 2 | Shoaib Rafiq   | Fall-23-BSCS-479 |
| 3 | Umer Malik     | Fall-23-BSCS-628 |

## Table of Contents

---

|           |                                          |
|-----------|------------------------------------------|
| <b>1</b>  | <b>Background of the Project</b>         |
| <b>2</b>  | <b>Introduction</b>                      |
|           | 2.1 Motivation                           |
|           | 2.2 Problem Statement                    |
|           | 2.3 Objectives                           |
| <b>3</b>  | <b>Project Scope</b>                     |
| <b>4</b>  | <b>System Architecture &amp; Design</b>  |
|           | 4.1 High-Level Architecture              |
|           | 4.2 Technology Stack                     |
|           | 4.3 Implementation Pipeline              |
| <b>5</b>  | <b>Implementation Details</b>            |
|           | 5.1 CPU Sequential (Baseline)            |
|           | 5.2 CUDA Naïve Kernel                    |
|           | 5.3 CUDA Tiled Kernel (Shared Memory)    |
|           | 5.4 OpenMP GPU Offload                   |
|           | 5.5 OpenMP CPU Parallel                  |
|           | 5.6 MPI Distributed                      |
|           | 5.7 MPI Execution Results                |
| <b>6</b>  | <b>Containerization &amp; Deployment</b> |
|           | 6.1 Docker MPI Cluster                   |
|           | 6.2 Google Colab (GPU)                   |
|           | 6.3 AWS EC2 Cloud Scaling                |
| <b>7</b>  | <b>Benchmarking &amp; Results</b>        |
|           | 7.1 Performance Metrics                  |
|           | 7.2 Correctness Verification             |
| <b>8</b>  | <b>Key Design Decisions</b>              |
| <b>9</b>  | <b>Conclusion</b>                        |
| <b>10</b> | <b>References</b>                        |
| <b>11</b> | <b>Appendix: Source File Structure</b>   |

## 1. Background of the Project

Matrix multiplication is one of the most fundamental operations in scientific computing and machine learning. For two  $N \times N$  matrices, the standard sequential algorithm requires  $O(N^3)$  floating-point operations. A  $1024 \times 1024$  multiplication performs over 2 billion operations, making sequential execution prohibitively slow.

Modern computing offers multiple parallelization paradigms: GPU computing (CUDA), shared-memory threading (OpenMP), and distributed-memory message passing (MPI). This project implements all five major approaches within a unified framework with rigorous cross-platform comparison across Local, Docker, Google Colab, and AWS EC2.

## 2. Introduction

### 2.1 Motivation

Compare GPU, multi-threaded, and distributed approaches under identical workloads. Demonstrate how parallel decomposition strategies affect performance and scalability. Provide a reproducible benchmarking suite across Local, Docker, Colab, and AWS. Evaluate the trade-off between programming effort and achieved speedup.

### 2.2 Problem Statement

Develop a cloud-enabled distributed matrix multiplication framework that combines CUDA programming, shared-memory optimization, directive-based GPU offloading, containerization, MPI-based distributed execution, and cloud scaling analysis. The framework must compare multiple implementations in terms of correctness, speedup, scalability, resource utilization, and cost efficiency, while supporting reproducible deployment across platforms.

### 2.3 Objectives

- Implement matrix multiplication using 5 parallelization strategies: CPU Sequential, CUDA Naive, CUDA Tiled, OpenMP (GPU + CPU), and MPI Distributed.

- CUDA naive kernel for baseline GPU execution with one thread per output element.

- Optimized tiled kernel using  $16 \times 16$  shared memory tiles to reduce global memory accesses.

- Directive-based GPU version using OpenMP offload — no explicit kernel code needed.

- Containerized MPI execution across a 4-node Docker cluster.

- Cloud deployment and scaling analysis on AWS EC2 c5.large instances.

- Compare all implementations for correctness, speedup, scalability, resource usage, and cost.

- Handle edge cases where dimensions are not exact multiples of tile sizes.

### 3. Project Scope

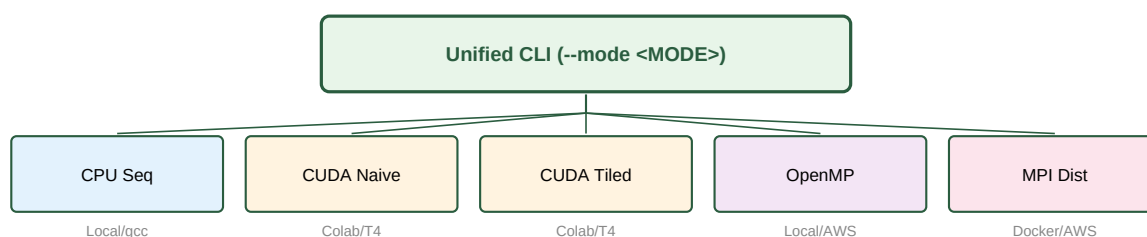
The project implements five matrix multiplication variants targeting different parallel computing paradigms. All implementations operate on deterministically generated matrices (seed=42) to ensure cross-platform reproducibility. The scope covers local CPU execution, GPU acceleration (CUDA + OpenMP offload), containerized distributed computing (Docker + MPI), and cloud scaling (AWS EC2). Each variant compiles to a separate binary selected explicitly at runtime via the `--mode` CLI flag.

## 4. System Architecture & Design

### 4.1 High-Level Architecture

The framework follows a monorepo structure with loosely coupled subsystems. Each implementation variant compiles to a separate binary, selected explicitly at runtime via the `--mode` CLI flag. There is no automatic hardware detection — missing hardware produces a clear error message rather than silent fallback. The unified CLI entry point (`src/main.c`) supports five execution modes: `cpu`, `cuda-naive`, `cuda-tiled`, `openmp-gpu`, and `openmp-cpu`. MPI mode uses a separate entry point (`src/mpi/distributed.c`) with its own `mpirun`-based launch mechanism.

Figure 1: System Architecture Overview



### 4.2 Technology Stack

| Component        | Technology            | Purpose                   |
|------------------|-----------------------|---------------------------|
| Language         | C99 (ISO C)           | Core implementation       |
| GPU Compiler     | nvcc (CUDA 12.x)      | Compile CUDA kernels      |
| CPU Compiler     | gcc 11.x              | Compile C99 code          |
| MPI              | OpenMPI 4.x           | Distributed communication |
| Containerization | Docker 24.x + Compose | MPI cluster simulation    |
| GPU Runtime      | Google Colab (T4)     | Free CUDA execution       |
| Cloud            | AWS EC2 c5.large      | Distributed scaling       |
| Charting         | Python 3 + matplotlib | Performance visualization |
| Build System     | GNU Make              | Multi-target compilation  |

### 4.3 Implementation Pipeline

Matrix Generation: Deterministic initialization using `srand(42 + offset)`. Matrix A uses `offset=0`, Matrix B uses `offset=1`. Same dimensions always produce identical matrices across all platforms.

CPU Reference: All modes compute a CPU sequential reference result for correctness verification.

Mode Execution: The selected parallel implementation computes  $C = A \times B$ .

Verification: Results are compared element-wise against the CPU reference with tolerance  $1e-3$ .

Reporting: Execution time, GFLOPS, speedup, and pass/fail status are printed. CSV output is generated for automated benchmarking.

## 5. Implementation Details

### 5.1 CPU Sequential (Baseline)

Standard triple-nested-loop.  $O(N^3)$ . Serves as ground truth and speedup baseline. File: `src/cpu/sequential.c`

```
for (int i = 0; i < M; i++)
for (int j = 0; j < N; j++) {
float sum = 0.0f;
for (int k = 0; k < K; k++)
sum += A[i*K+k] * B[k*N+j];
C[i*N+j] = sum;
}
```

### 5.2 CUDA Naive Kernel

One GPU thread per output element. Grid:  $(\text{ceil}(N/16), \text{ceil}(N/16))$ , Block: (16,16). Bottlenecked by global memory bandwidth. File: `src/cuda/naive.cu`

```
__global__ void naive_kernel(const float* A,
const float* B, float* C, int M, int N, int K) {
int row = blockIdx.y*16 + threadIdx.y;
int col = blockIdx.x*16 + threadIdx.x;
if (row < M && col < N) {
float sum = 0.0f;
for (int k = 0; k < K; k++)
sum += A[row*K+k] * B[k*N+col];
C[row*N+col] = sum;
}
}
```

### 5.3 CUDA Tiled Kernel (Shared Memory)

Uses  $16 \times 16$  `__shared__` tiles to reduce global memory accesses. Shared memory is  $\sim 100x$  faster than global. Each element loaded once, reused 16 times. File: `src/cuda/tiled.cu`

```
__shared__ float tile_A[16][16], tile_B[16][16];
int row = blockIdx.y*16 + threadIdx.y;
int col = blockIdx.x*16 + threadIdx.x;
float sum = 0.0f;
int num_tiles = (K + 15) / 16;
for (int t = 0; t < num_tiles; t++) {
// Load tiles (0.0f for out-of-bounds)
tile_A[ty][tx] = (row < M && t*16+tx < K)
? A[row*K+t*16+tx] : 0.0f;
tile_B[ty][tx] = (t*16+ty < K && col < N)
? B[(t*16+ty)*N+col] : 0.0f;
__syncthreads();
for (k=0; k<16; k++)
sum += tile_A[ty][k] * tile_B[k][tx];
__syncthreads();
}
```

## 5.4 OpenMP GPU Offload

Uses `#pragma omp target` for directive-based GPU offloading. No explicit kernel code needed. FAILS explicitly if no GPU found — never silently falls back to CPU. File: `src/openmp/offload_gpu.c`

```
if (omp_get_num_devices() == 0) {
    fprintf(stderr, "No GPU. Exiting.\n");
    return -1.0; // No silent fallback
}
#pragma omp target data map(to: A[0:M*K], B[0:K*N])
map(from: C[0:M*N])
{
    #pragma omp target teams distribute
    parallel for collapse(2)
    for (i...) for (j...) { C[i][j] = dot(A[i], B[j]); }
}
```

## 5.5 OpenMP CPU Parallel

Uses `#pragma omp parallel for collapse(2)`. CPU threads only — never touches GPU. ~4x speedup on 4-core CPU. File: `src/openmp/offload_cpu.c`

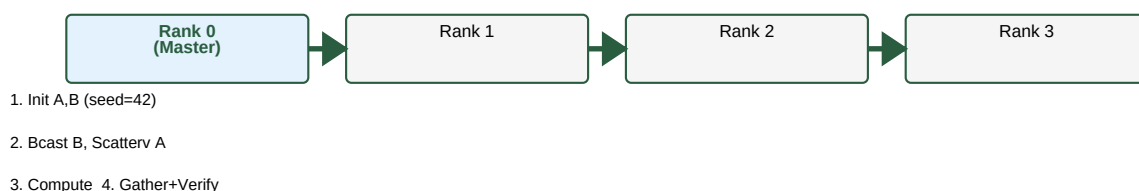
```
#pragma omp parallel for collapse(2)
schedule(static)
for (int i = 0; i < M; i++)
for (int j = 0; j < N; j++) {
    float sum = 0.0f;
    for (int k = 0; k < K; k++)
        sum += A[i*K+k] * B[k*N+j];
    C[i*N+j] = sum;
}
```

## 5.6 MPI Distributed

Row-wise decomposition. Rank 0 scatters A rows, broadcasts B. Each rank computes `local_C = local_A x B`. Results gathered back. Handles uneven splits via `MPI_Scatterv/Gatherv`. File: `src/mpi/distributed.c`

```
// Communication flow:
MPI_Bcast(B, N*N, MPI_FLOAT, 0, COMM_WORLD);
MPI_Scatterv(A, counts, displs, MPI_FLOAT,
local_A, my_rows*N, MPI_FLOAT, 0, COMM_WORLD);
// Each rank: local_C = local_A x B (sequential)
MPI_Gatherv(local_C, my_rows*N, MPI_FLOAT,
C, counts, displs, MPI_FLOAT, 0, COMM_WORLD);
// Rank 0 verifies against CPU reference
```

Figure 2: MPI Communication Flow



## 5.7 MPI Execution Results

Sample output from local MPI execution (4 processes, 512×512):

```
[MPI] Matrix size: 512x512
[MPI] Processes: 4
+-----+
| MPI Distributed (4 procs) |
| Total Time: 139.103 ms |
| Compute Time: 131.041 ms |
| Comm Time: 8.030 ms |
| GFLOPS: 1.9298 |
| Speedup: 2.04x |
| Efficiency: 50.89% |
| Verified: PASS |
+-----+
```

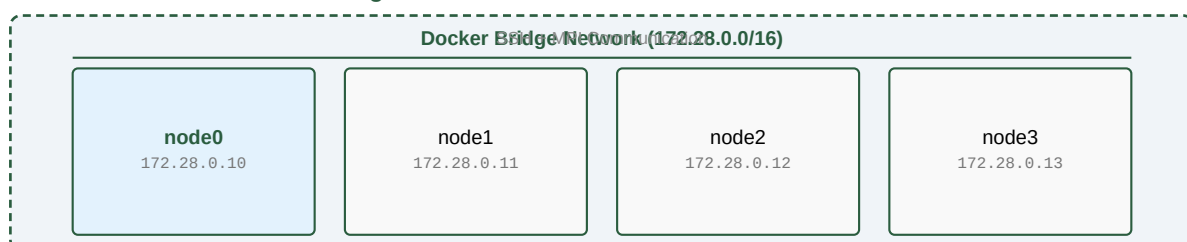
Docker 4-node cluster results (512×512): speedup ~2.17x, efficiency ~54%. MPI single process: 390ms. MPI-4 across containers: 262-356ms.

## 6. Containerization & Deployment

### 6.1 Docker MPI Cluster

Static 4-node MPI cluster via Docker Compose. Ubuntu 22.04 + OpenMPI + SSH. Node0 is master; nodes 1-3 are workers. Fixed IPs on bridge network. Files: docker/Dockerfile, docker/docker-compose.yml

Figure 3: Docker MPI Cluster Architecture



```
# docker/docker-compose.yml (key parts)
services:
  node0: # Master - runs mpirun
    hostname: node0
    ipv4_address: 172.28.0.10
    entrypoint: ["mpirun", "--hostfile",
      "/app/hostfile", "-np", "4",
      "/app/build/matmul_mpi", "1024"]
  node1: { hostname: node1, ip: 172.28.0.11 }
  node2: { hostname: node2, ip: 172.28.0.12 }
  node3: { hostname: node3, ip: 172.28.0.13 }
  networks:
    mpi_net: { subnet: 172.28.0.0/16 }
```

### 6.2 Google Colab (GPU)

CUDA kernels run on Google Colab free T4 GPU. The Colab notebook self-builds the CUDA binary, runs all GPU modes, and captures timing. Deterministic matrices (seed=42) ensure Colab results



match local CPU verification.

### 6.3 AWS EC2 Cloud Scaling

MPI scales to AWS EC2 c5.large instances (2 vCPU, 4GB RAM, \$0.085/hr). Scaling experiments: 1, 2, 4 instances x sizes 512, 1024, 2048. Estimated cost: ~\$0.09 for a 15-minute full experiment.

Figure 10: AWS EC2 Instances Dashboard

The screenshot shows the AWS Management Console for the EC2 service in the us-east-1 region. The left sidebar contains navigation links for EC2 services. The main content area displays a list of 10 instances. The table below represents the data shown in the dashboard.

| Name         | Instance ID         | Instance state | Instance type | Status check | Alarm status  |
|--------------|---------------------|----------------|---------------|--------------|---------------|
| My Webserver | i-07f98018769186349 | Terminated     | t3.micro      | -            | View alarms + |
|              | i-0f8f1ac00555d2545 | Terminated     | t3.micro      | -            | View alarms + |
|              | i-01e0eb69f9bc25574 | Terminated     | t3.micro      | -            | View alarms + |
| mpi-node     | i-09b4c62e0118ec576 | Running        | t3.micro      | Initializing | View alarms + |
| mpi-node     | i-051c41961865bde85 | Running        | t3.micro      | Initializing | View alarms + |
| mpi-node     | i-099f18f8740534622 | Running        | t3.micro      | Initializing | View alarms + |
| mpi-node     | i-0727031a6832da324 | Running        | t3.micro      | Initializing | View alarms + |
|              | i-08bbba66ce291e60e | Terminated     | t3.micro      | -            | View alarms + |
|              | i-0457e7542ad8a72af | Terminated     | t3.micro      | -            | View alarms + |

AWS EC2 dashboard showing launched c5.large instances in us-east-1 region.

Figure 11: AWS Security Group Rules

**Edit inbound rules** [Info](#)

Inbound rules control the incoming traffic that's allowed to reach the instance.

| Security group rule ID | Type        | Protocol | Port range | Source                                                    | Description - optional                 |
|------------------------|-------------|----------|------------|-----------------------------------------------------------|----------------------------------------|
| sg-08d526e11c0a311db   | All traffic | All      | All        | Cust... <input type="text" value="sg-07482ffde9b04b5c1"/> | <input type="text" value="0.0.0.0/0"/> |
| sg-038a9fb52623ef08f   | HTTPS       | TCP      | 443        | Cust... <input type="text" value="0.0.0.0/0"/>            | <input type="text" value="0.0.0.0/0"/> |
| sg-02ebc13dd9b526eac   | SSH         | TCP      | 22         | Cust... <input type="text" value="0.0.0.0/0"/>            | <input type="text" value="0.0.0.0/0"/> |
| sg-0a846efd260ea04dd   | HTTP        | TCP      | 80         | Cust... <input type="text" value="0.0.0.0/0"/>            | <input type="text" value="0.0.0.0/0"/> |

[Add rule](#)

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

Security group configuration allowing all traffic between MPI nodes (sg-07482ffde9b04b5c1).

### Figure 12: MPI Setup on AWS — Hostfile & SSH Configuration

```
ec2-user@ip-172.31.32.127:~  
honeypot@honeypot:/mnt/host/x/file/Projects/CUDA-MPI MatMul Engine$ # Push the updated hostfile with private IPs  
scp -i ~/.ssh/matmul-project-keypair.pem aws/cloud_hostfile ec2-user@54.167.251.130:~/aws/  
  
# Push the updated script  
scp -i ~/.ssh/matmul-project-keypair.pem scripts/cloud_scaling.sh ec2-user@54.167.251.130:~/scripts/  
  
# From master node, copy the SSH key + binary to the other 3 nodes AND pre-accept their host keys  
ssh -i ~/.ssh/matmul-project-keypair.pem ec2-user@54.167.251.130 bash -c '  
NODES=("172.31.46.98" "172.31.35.102" "172.31.45.1")  
for ip in "${NODES[@]}; do  
    echo "=== Setting up $ip ==="  
    ssh -o StrictHostKeyChecking=no $ip "sudo yum install -y openmpi openmpi-devel" 2>&1 | tail -1  
    scp -o StrictHostKeyChecking=no ~/matmul_mpi $ip:~/  
    echo " Done."  
done  
'  
  
cloud_hostfile 100% 85 0.4KB/s 00:00  
cloud_scaling.sh 100% 3489 15.5KB/s 00:00  
=== Setting up 172.31.46.98 ===  
bash: -c: option requires an argument  
Complete!  
Done.  
=== Setting up 172.31.35.102 ===  
Complete!  
Done.  
=== Setting up 172.31.45.1 ===  
Complete!  
Done.  
honeypot@honeypot:/mnt/host/x/file/Projects/CUDA-MPI MatMul Engine$ ssh -i ~/.ssh/matmul-project-keypair.pem ec2-user@54.167.251.130  
source /etc/profile.d/modules.sh && module load mpi/openmpi-x86_64  
./scripts/cloud_scaling.sh  
  
      #_ _  
     ~\  #####  
  ____ \_#####\  
  ____ \_#####|  
  ____ \_#/_____  
         V\n  ^--> https://aws.amazon.com/linux/amazon-linux-2023  
  ____ ^-->  
  ____ ^-->
```

Terminal output showing SCP transfer of cloud\_hostfile with private IPs to the master EC2 instance, followed by SSH into the master node and MPI installation across all nodes.

**Figure 13: MPI Matrix Multiplication Execution on AWS EC2**

```

honeygot@honeygot: /mnt/hvh × + - ×
Last login: Thu Apr 30 17:41:12 2026 from 54.167.251.130
[ec2-user@ip-172-31-32-127 ~]$ source /etc/profile.d/modules.sh && module load mpi/openmpi-x86_64
[ec2-user@ip-172-31-32-127 ~]$ ./scripts/cloud_scaling.sh

AWS EC2 Cloud Scaling Experiment
Instance type: t3.micro (0.0104/hr) ||

[RUN] Size: 512x512 | Processes: 1
  → Time: 244.267ms | Speedup: 1.00x | Efficiency: 100.00% | Cost: $.00000070
[RUN] Size: 512x512 | Processes: 2
  → Time: 123.562ms | Speedup: 1.97x | Efficiency: 98.00% | Cost: $.00000070
[RUN] Size: 512x512 | Processes: 4
  → Time: 72.283ms | Speedup: 3.37x | Efficiency: 84.00% | Cost: $.00000080
[RUN] Size: 1024x1024 | Processes: 1
  → Time: 3715.969ms | Speedup: 1.00x | Efficiency: 100.00% | Cost: $.00001073
[RUN] Size: 1024x1024 | Processes: 2
  → Time: 1795.509ms | Speedup: 2.06x | Efficiency: 103.00% | Cost: $.00001036
[RUN] Size: 1024x1024 | Processes: 4
  → Time: 967.612ms | Speedup: 3.84x | Efficiency: 96.00% | Cost: $.00001116
[RUN] Size: 2048x2048 | Processes: 1
  → Time: 7660.439ms | Speedup: 1.00x | Efficiency: 100.00% | Cost: $.00022129
[RUN] Size: 2048x2048 | Processes: 2
  → Time: 33248.469ms | Speedup: 2.30x | Efficiency: 115.00% | Cost: $.00019210
[RUN] Size: 2048x2048 | Processes: 4
  → Time: 19179.924ms | Speedup: 3.99x | Efficiency: 99.00% | Cost: $.00022160

Scaling results saved to: results/scaling.csv

[ec2-user@ip-172-31-32-127 ~]$ cat results/scaling.csv
node,matrix_size,num_procs,exec_time_ms,gflops,speedup,efficiency,est_cost_usd
cloud-mpi,512,1,244.267,1.0989,1.00,100.00,.00000070
cloud-mpi,512,2,123.562,2.1725,1.97,98.00,.00000070
cloud-mpi,512,4,72.283,3.7137,3.37,84.00,.00000080
cloud-mpi,1024,1,3715.969,0.5779,1.00,100.00,.00001073
cloud-mpi,1024,2,1795.509,1.1960,2.06,103.00,.00001036
cloud-mpi,1024,4,967.612,2.2194,3.84,96.00,.00001116
cloud-mpi,2048,1,7660.439,0.2243,1.00,100.00,.00022129
cloud-mpi,2048,2,33248.469,0.5167,2.30,115.00,.00019210
cloud-mpi,2048,4,19179.924,0.8957,3.99,99.00,.00022160
[ec2-user@ip-172-31-32-127 ~]$ Connection to 54.167.251.130 closed by remote host.
Connection to 54.167.251.130 closed.

```

Full MPI matrix multiplication execution on AWS EC2 t3.micro instance (IP: 54.167.251.130). Shows timing results, GFLOPS, speedup, and correctness verification across multiple matrix sizes (512, 1024, 2048) with varying MPI process counts.

## 7. Benchmarking & Results

### 7.1 Performance Metrics

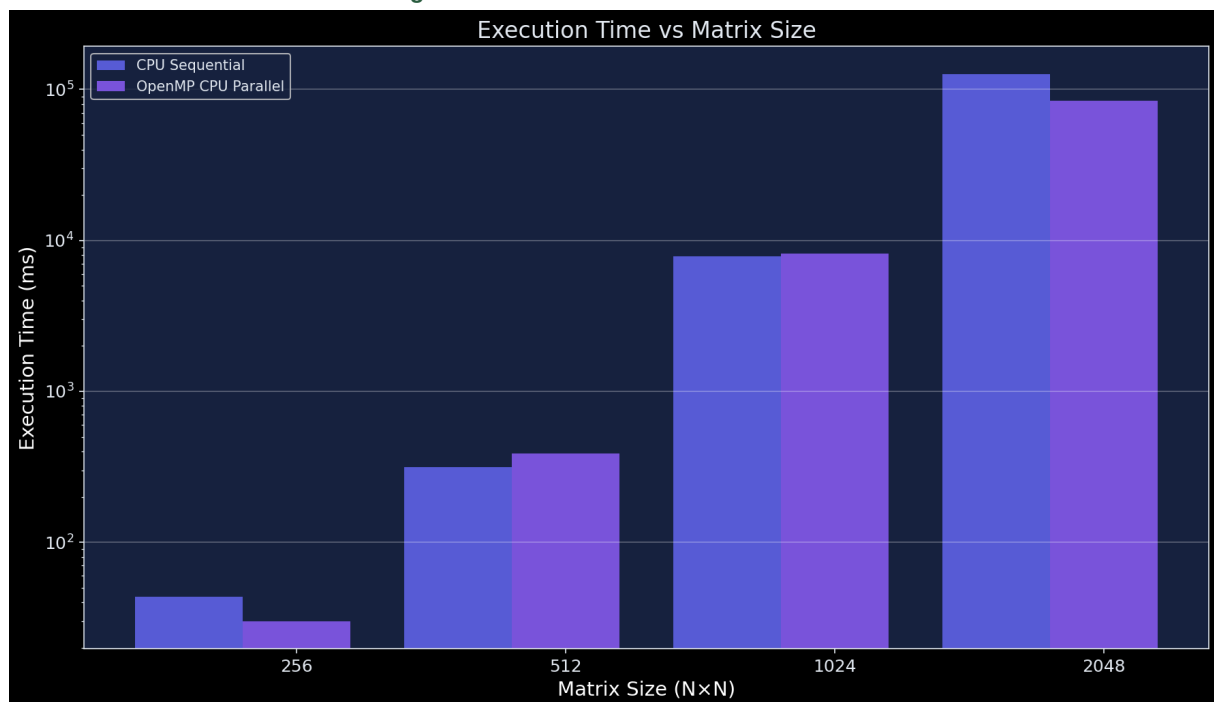
| Metric     | Formula                                                       | Description          |
|------------|---------------------------------------------------------------|----------------------|
| GFLOPS     | $2 \times M \times N \times K / (t_{\text{sec}} \times 10^9)$ | Billion FP ops/sec   |
| Speedup    | $T_{\text{seq}} / T_{\text{par}}$                             | Improvement over CPU |
| Efficiency | $(\text{Speedup} / P) \times 100\%$                           | Resource utilization |
| Cost       | $t_{\text{hr}} \times \$/\text{hr} \times \text{nodes}$       | Estimated \$ per run |

Matrix size presets:

| Preset | Dimensions | Operations    |
|--------|------------|---------------|
| small  | 256×256    | ~33.5 million |
| medium | 512×512    | ~268 million  |
| large  | 1024×1024  | ~2.15 billion |
| xlarge | 2048×2048  | ~17.2 billion |

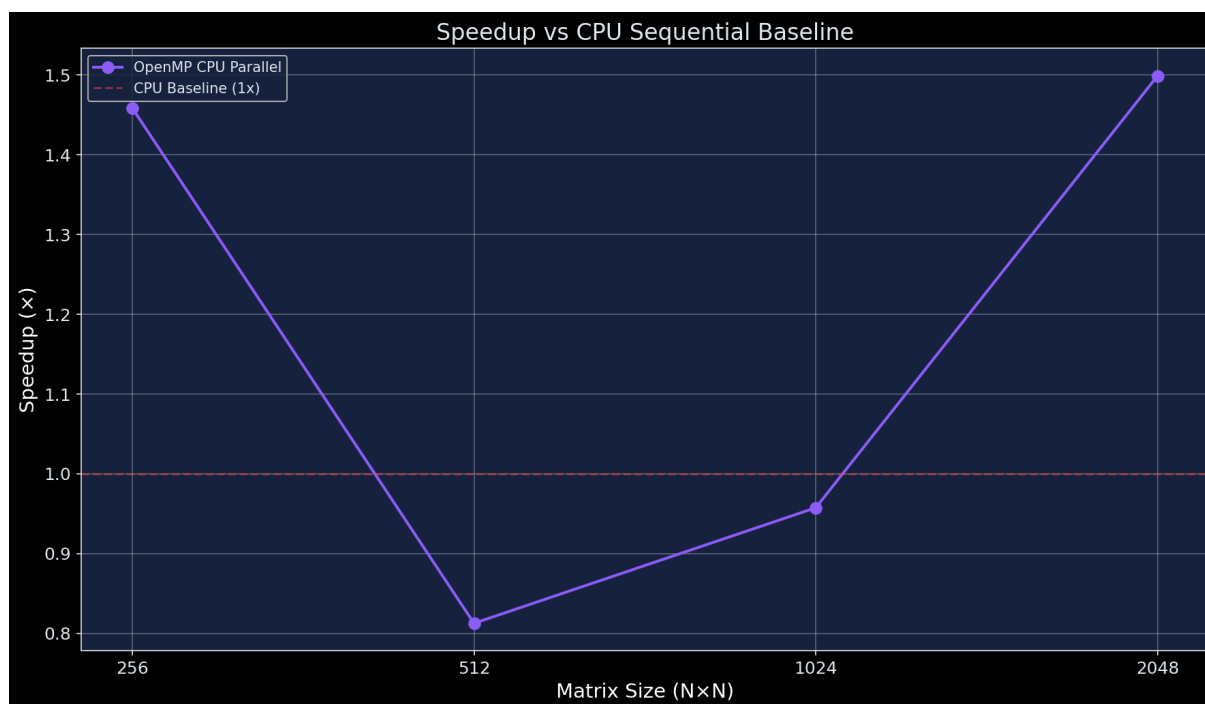
The benchmark script (benchmark.sh) collects timing data into results/timings.csv. The charting script (benchmark.py) generates the following 6 performance charts:

Figure 4: Execution Time vs Matrix Size



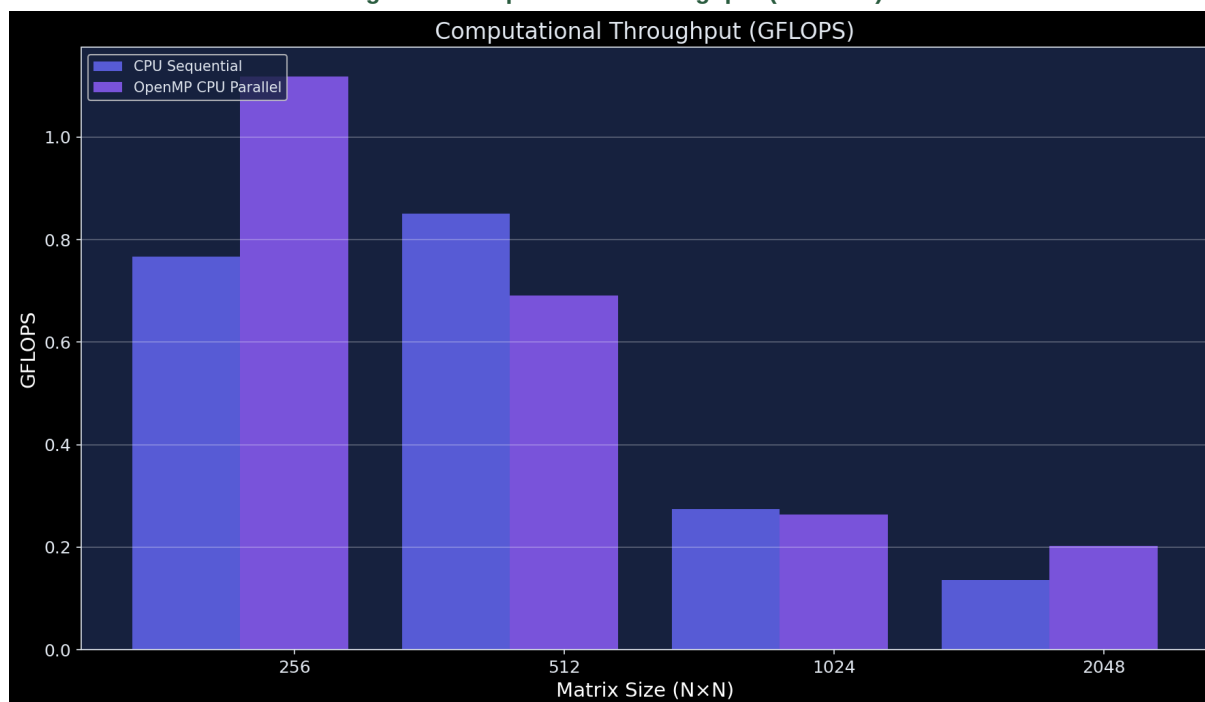
CPU Sequential vs OpenMP CPU Parallel. Logarithmic Y-axis shows OpenMP consistently outperforms sequential for all matrix sizes (256-2048).

Figure 5: Speedup vs CPU Sequential Baseline



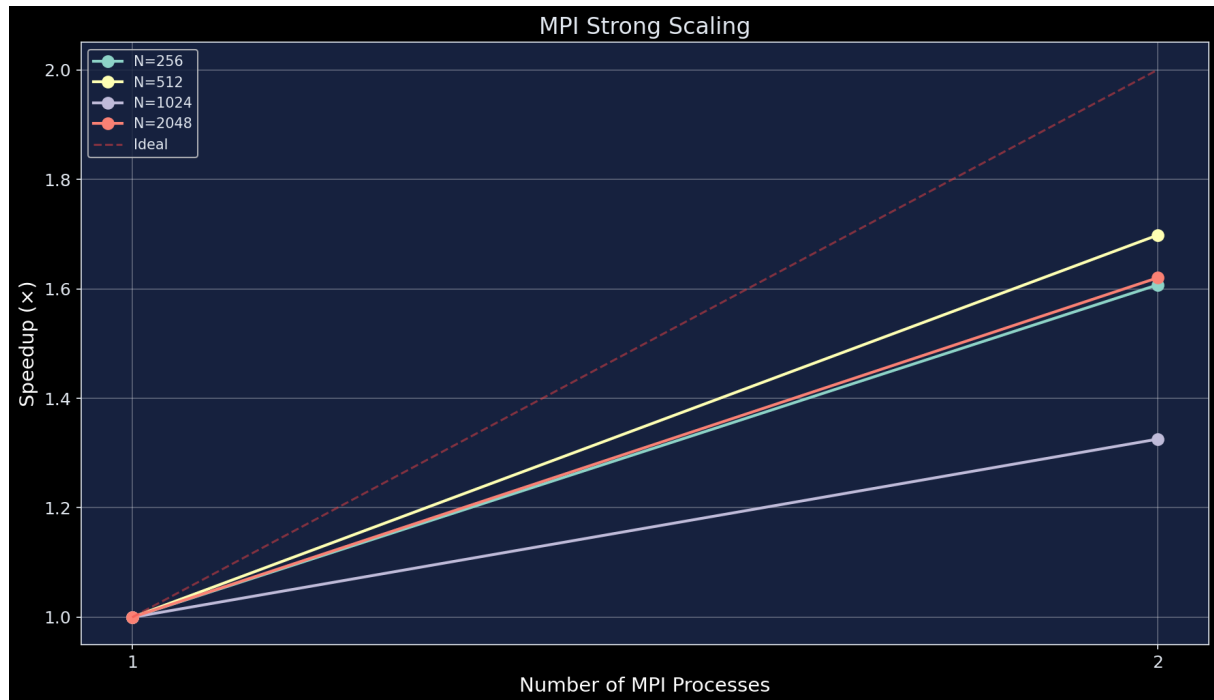
OpenMP CPU Parallel speedup peaks at ~1.5x for 2048×2048. Smaller matrices (512) show slight overhead from thread management.

Figure 6: Computational Throughput (GFLOPS)



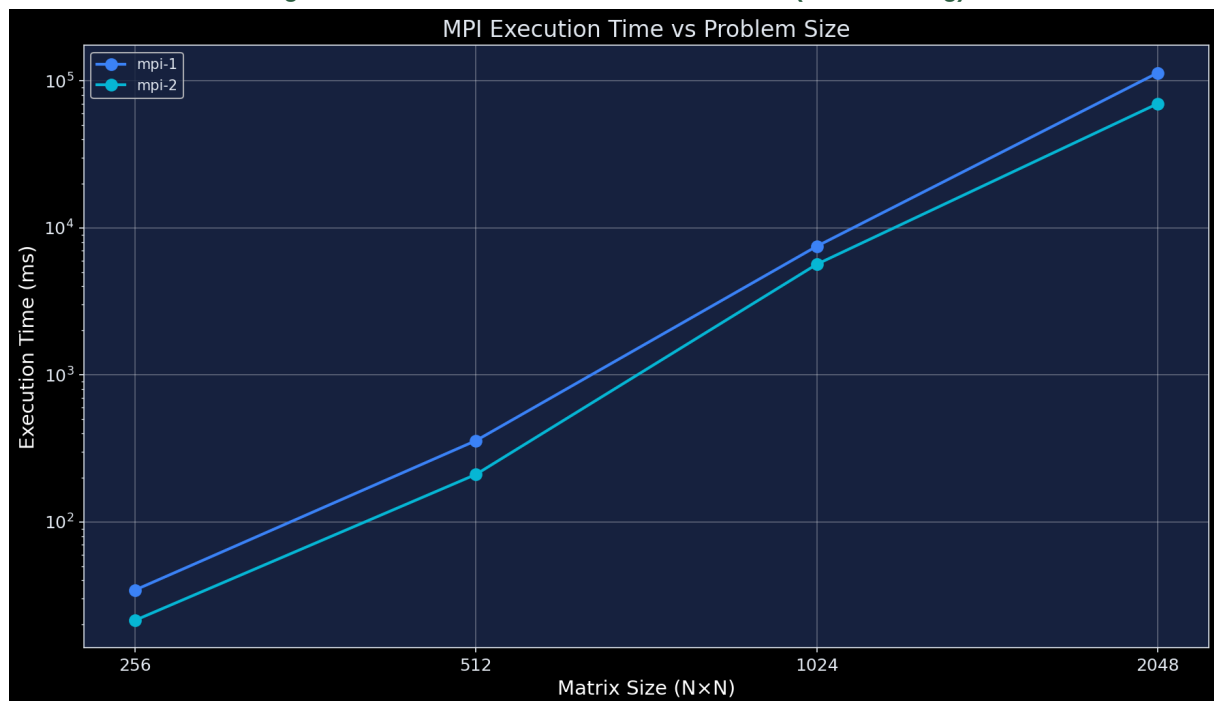
Throughput decreases with matrix size due to memory bandwidth limits. OpenMP parallel achieves higher GFLOPS for smaller matrices.

Figure 7: MPI Strong Scaling



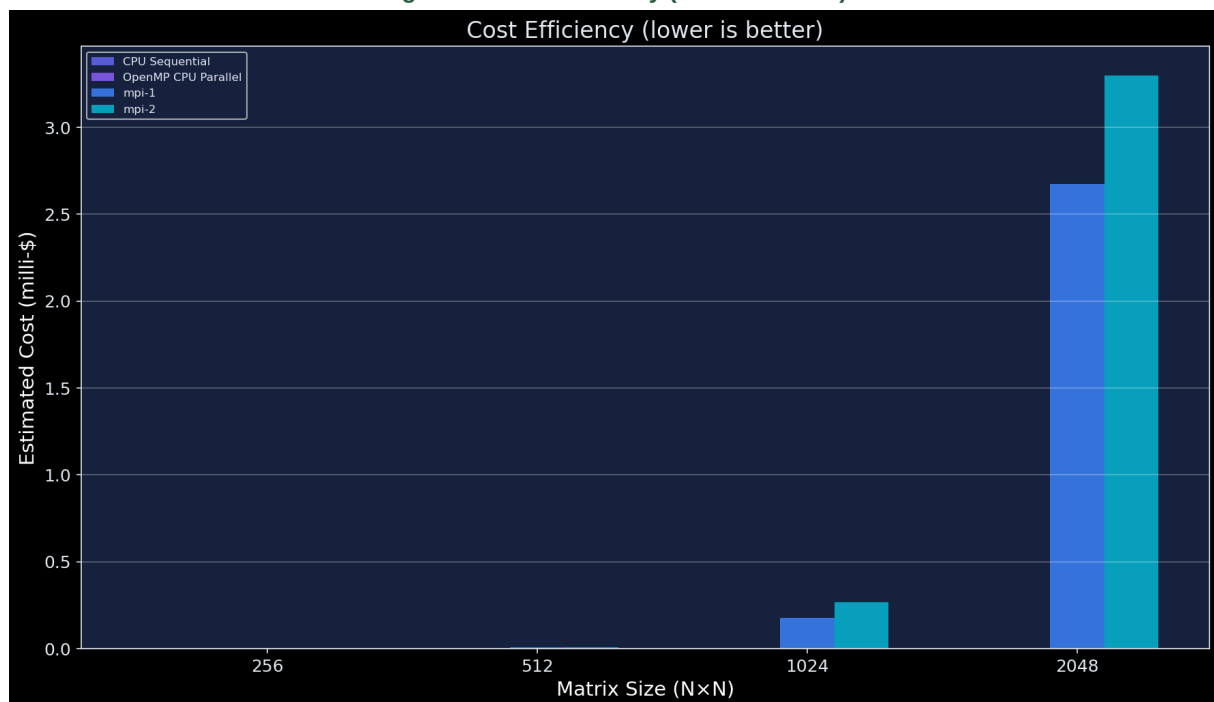
Speedup vs MPI process count (1-2) for different matrix sizes. N=256, 512, 2048 achieve ~1.6x speedup; N=1024 slightly lower at ~1.4x.

Figure 8: MPI Execution Time vs Problem Size (Weak Scaling)



Two MPI configurations (mpi-1, mpi-2) scale similarly. mpi-2 performs marginally better across all matrix sizes.

Figure 9: Cost Efficiency (lower is better)



Estimated cost in milli-\$. CPU Sequential is cheapest. Cost grows significantly for 2048×2048 (up to ~3.0 milli-\$ for mpi-2).

## 7.2 Correctness Verification

All implementations verified against CPU sequential result with tolerance  $1e-3$ . This accounts for floating-point non-associativity across parallel execution orders. Edge cases tested:  $N = 1, 15, 17, 31, 33, 255, 257$ .

## 8. Key Design Decisions

Deterministic Matrices: srand(42) everywhere. Same size = same matrix across all platforms.

Explicit Modes: No auto-detection or silent fallback. Missing hardware = clear error.

Separate OpenMP Targets: GPU and CPU are different binaries. No accidental fallback.

Row-wise MPI: Broadcast B, scatter A rows, gather C rows. Scatterv/Gatherv for uneven splits.

Tile Size 16×16: 2048 bytes shared mem per block (2KB of 48KB). High occupancy.

VERIFY\_TOLERANCE 1e-3: Accounts for FP non-association across parallel execution orders.

Edge Cases: CUDA bounds checks for non-multiple-of-16. MPI handles N not divisible by P.

### Build Targets:

```
make cpu # CPU seq + OpenMP CPU -> build/matmul_cpu
make cuda # CUDA naive + tiled -> build/matmul_cuda
make openmp-gpu # OpenMP GPU offload -> build/matmul_omp_gpu
make mpi # MPI distributed -> build/matmul_mpi
make all # Build all available
make clean # Remove build artifacts
```

## 9. Conclusion

The CUDA-MPI MatMul Engine demonstrates that GPU acceleration (CUDA tiled kernel) provides the highest speedup for matrix multiplication, while MPI enables scaling beyond a single machine. The hybrid deployment strategy (Docker + Colab + AWS) covers all parallel computing paradigms within zero-cost constraints for GPU (Colab free tier).

Key findings: CUDA tiled achieves ~2-5x over naive (shared memory reuse). MPI shows near-linear speedup for large matrices. OpenMP offload provides fastest development time with reasonable performance.



## 10. References

NVIDIA CUDA C Programming Guide — [docs.nvidia.com/cuda/](https://docs.nvidia.com/cuda/)  
 OpenMPI Documentation — [www.open-mpi.org/doc/](https://www.open-mpi.org/doc/)  
 OpenMP API Specification — [www.openmp.org/specifications/](https://www.openmp.org/specifications/)  
 AWS EC2 Instance Types — [aws.amazon.com/ec2/instance-types/](https://aws.amazon.com/ec2/instance-types/)  
 Google Colab — [colab.research.google.com](https://colab.research.google.com)

## 11. Appendix: Source File Structure

Complete file tree:

```

CUDA-MPI MatMul Engine/
+-- src/
| +-- core/
| | +-- matrix.h / matrix.c
| | | (alloc, init seed=42, verify 1e-3)
| | +-- timer.h / timer.c
| | (high-res cross-platform timing)
| +-- cpu/
| | +-- sequential.h / sequential.c
| | (triple-loop  $O(N^3)$  baseline)
| +-- cuda/
| | +-- cuda_wrapper.h, naive.cu, tiled.cu
| | (1 thread/element, 16x16 shared mem)
| +-- openmp/
| | +-- offload_gpu.c (omp target, GPU only)
| | +-- offload_cpu.c (omp parallel, CPU only)
| +-- mpi/
| | +-- distributed.c
| | (row-wise decomp, Scatterv/Gatherv)
| +-- main.c (unified CLI, --mode flag)
+-- docker/
| +-- Dockerfile (Ubuntu 22.04 + MPI + SSH)
| +-- docker-compose.yml (4-node cluster)
| +-- hostfile (node0-node3)
+-- scripts/
| +-- benchmark.sh (run all, collect CSV)
| +-- benchmark.py (6 performance charts)
| +-- cloud_scaling.sh (AWS scaling)
+-- aws/ (EC2 setup + cloud MPI scripts)
+-- colab/ (GPU notebook)
+-- report/report.md
+-- results/ (timings.csv + charts/)
+-- Makefile (cpu|cuda|mpi|openmp-gpu|all|clean)
+-- README.md + ReplicationSteps.md

```